

Vertex AI：搭配 Sklearn 使用自訂預測處理常式，預先處理及後續處理資料以執行預測

來源：<https://codelabs.developers.google.com/codelabs/vertex-cpr-sklearn?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將瞭解如何在 Vertex AI 中使用自訂預測處理常式，編寫自訂預先處理和後續處理邏輯

目錄

1. [簡介](#)
2. [Vertex AI 簡介](#)
3. [用途總覽](#)
4. [啟用教學課程 API](#)
5. [建立 aiml-vpc](#)
6. [建立由使用者管理的筆記本](#)
7. [編寫訓練程式碼](#)
8. [使用 CPR 模型伺服器建構自訂服務容器](#)
9. [將模型部署至 Vertex AI](#)

步驟 1 · 約 0 分鐘

Vertex AI：使用 Sklearn 的自訂預測處理常式，預先處理及後續處理預測資料

程式碼研究室簡介

*subject*上次更新時間：3月 27, 2026

*account_circle*作者：Nikita Namjoshi, Deepak Michael

1. 簡介

在本實驗室中，您將瞭解如何在 Vertex AI 上使用自訂預測處理常式，編寫自訂的預先處理和後續處理邏輯。雖然本範例使用 Scikit-learn，但自訂預測處理常式可搭配其他 Python 機器學習架構，例如 XGBoost、PyTorch 和 TensorFlow。

課程內容

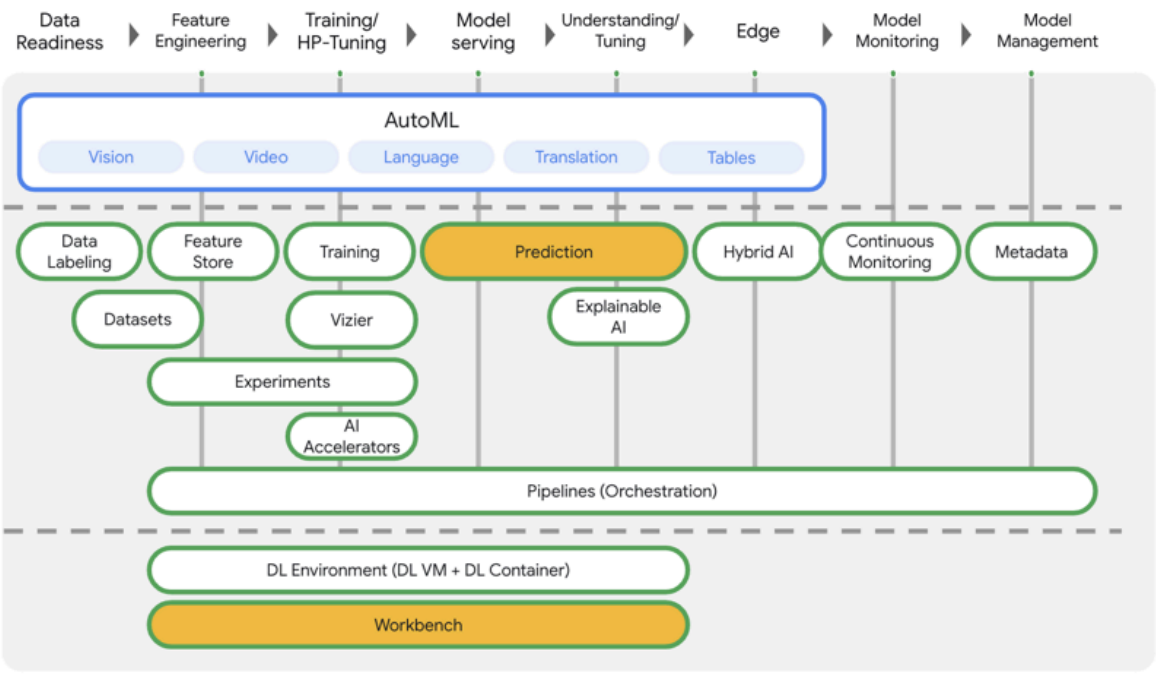
- 使用自訂預測處理常式撰寫自訂預測邏輯
 - 在本機測試自訂服務容器和模型
 - 在 Vertex AI Predictions 上測試自訂服務容器
-

步驟 2 · 約 2 分鐘

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於「預測」和「工作台」。



3. 用途總覽

在本實驗室中，您將建構隨機森林迴歸模型，根據切工、淨度和大小等屬性預測鑽石價格。

您將編寫自訂預先處理邏輯，檢查服務時的資料是否為模型預期的格式。您也會編寫自訂後續處理邏輯，將預測結果四捨五入並轉換為字串。如要編寫這項邏輯，請使用自訂預測處理常式。

自訂預測處理常式簡介

Vertex AI 預先建構的容器會執行機器學習架構的預測作業，處理預測要求。在自訂預測處理常式推出前，如果您想在執行預測前預先處理輸入內容，或在傳回結果前對模型預測進行後處理，就必須建構自訂容器。

如要建構自訂服務容器，必須編寫 HTTP 伺服器，將訓練好的模型包裝起來、將 HTTP 要求轉換為模型輸入內容，並將模型輸出內容轉換為回應。

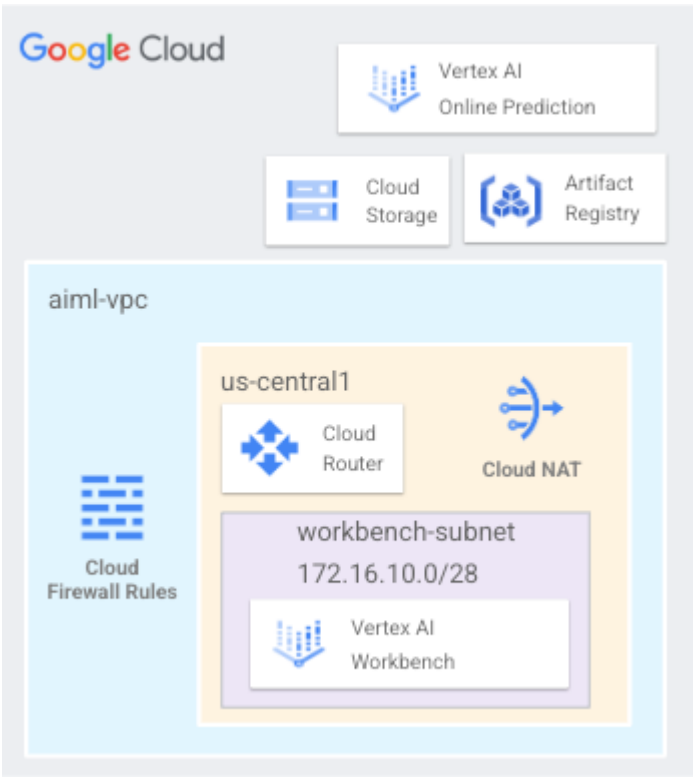
透過自訂預測處理常式，Vertex AI 會為您提供服務相關元件，讓您專注於模型和資料轉換。

建構項目

您將設定名為 aiml-vpc 的虛擬私有雲網路，其中包含用於部署使用者管理筆記本的工作台子網路，並存取在 us-central1 中部署的線上預測和模型端點，如下圖 1 所示。

○○○

Figure1



步驟 4 · 約 5 分鐘

4. 啟用教學課程 API

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」(如果尚未啟用)。您需要這項資訊才能建立筆記本執行個體。

步驟 2：啟用 Artifact Registry API

前往 [Artifact Registry](#)，然後選取「啟用」(如果尚未啟用)。您將使用這個工具建立自訂服務容器。

步驟 3：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後按一下「啟用 Vertex AI API」。

步驟 4：建立 Vertex AI Workbench 執行個體

如果尚未啟用，請啟用 [Notebooks API](#)。

步驟 5 · 約 10 分鐘

5. 建立 aiml-vpc

本教學課程會使用 \$variables，協助您在 Cloud Shell 中實作 gcloud 設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

建立 aiml-vpc

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create aiml-vpc --project=$projectid --subnet-mode=custom
```

建立由使用者管理的筆記本子網路

在 Cloud Shell 中建立 workbench-subnet。

```
gcloud compute networks subnets create workbench-subnet --project=$projectid --
range=172.16.10.0/28 --network=aiml-vpc --region=us-central1 --enable-private-ip-google-
access
```

Cloud Router 和 NAT 設定

本教學課程會使用 Cloud NAT 下載軟體套件，因為使用者管理的筆記本沒有外部 IP 位址。Cloud NAT 提供連出 NAT 功能，也就是說，網際網路主機無法與使用者管理的 Notebook 啟動通訊，因此更加安全。

在 Cloud Shell 中，建立 us-central1 地區的 Cloud Router。

```
gcloud compute routers create cloud-router-us-central1-aiml-nat --network aiml-vpc --region
us-central1
```

在 Cloud Shell 中，建立區域性 Cloud NAT 閘道 (us-central1)。

```
gcloud compute routers nats create cloud-nat-us-central1 --router=cloud-router-us-central1-
aiml-nat --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges --region us-central1
```

6. 建立由使用者管理的筆記本

建立使用者管理的服務帳戶 (Notebook)

在下一節中，您將建立與本教學課程所用 Vertex Workbench (Notebook) 相關聯的使用者代管服務帳戶。

在本教學課程中，服務帳戶會套用下列規則：

- [儲存空間管理員](#)
- [Vertex AI 使用者](#)
- [Artifact Registry 管理員](#)

在 Cloud Shell 中建立服務帳戶。

```
gcloud iam service-accounts create user-managed-notebook-sa \  
  --display-name="user-managed-notebook-sa"
```

在 Cloud Shell 中，將服務帳戶更新為 Storage 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:user-managed-  
notebook-sa@$projectid.iam.gserviceaccount.com" --role="roles/storage.admin"
```

在 Cloud Shell 中，使用 Vertex AI 使用者角色更新服務帳戶。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:user-managed-  
notebook-sa@$projectid.iam.gserviceaccount.com" --role="roles/aiplatform.user"
```

在 Cloud Shell 中，更新服務帳戶，並指派 Artifact Registry 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:user-managed-  
notebook-sa@$projectid.iam.gserviceaccount.com" --role="roles/artifactregistry.admin"
```

在 Cloud Shell 中列出服務帳戶，並記下建立使用者管理的筆記本時要使用的電子郵件地址。

```
gcloud iam service-accounts list
```

建立由使用者管理的 Notebook

在下一節中，建立納入先前建立的服務帳戶 (user-managed-notebook-sa) 的使用者管理筆記本。

在 Cloud Shell 中建立 private-client 執行個體。

```
gcloud notebooks instances create workbench-tutorial \  
  --vm-image-project=deeplearning-platform-release \  
  --vm-image-family=common-cpu-notebooks \  
  --machine-type=n1-standard-4 \  
  --location=us-central1-a \  
  --shielded-secure-boot \  
  --subnet-region=us-central1 \  
  --subnet=workbench-subnet \  
  --no-public-ip    --service-account=user-managed-notebook-  
sa@$projectid.iam.gserviceaccount.com
```

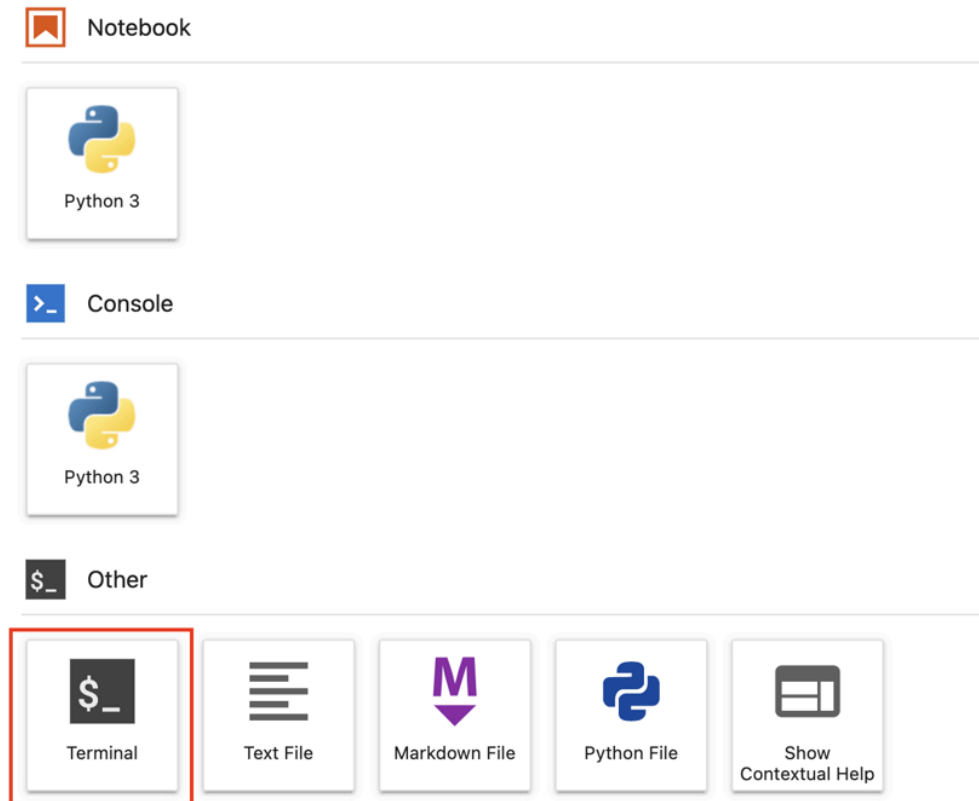
步驟 7 · 約 10 分鐘

7. 編寫訓練程式碼

步驟 1：建立 Cloud Storage bucket

您會將模型和前處理構件儲存在 Cloud Storage bucket 中。如果您已在專案中建立要使用的 bucket，可以略過這個步驟。

從啟動器開啟新的終端機工作階段。



在終端機中執行下列指令，為專案定義環境變數，並務必將 `your-cloud-project` 替換為專案 ID：

```
PROJECT_ID='your-cloud-project'
```

接著，在終端機中執行下列指令，在專案中建立新的 bucket。

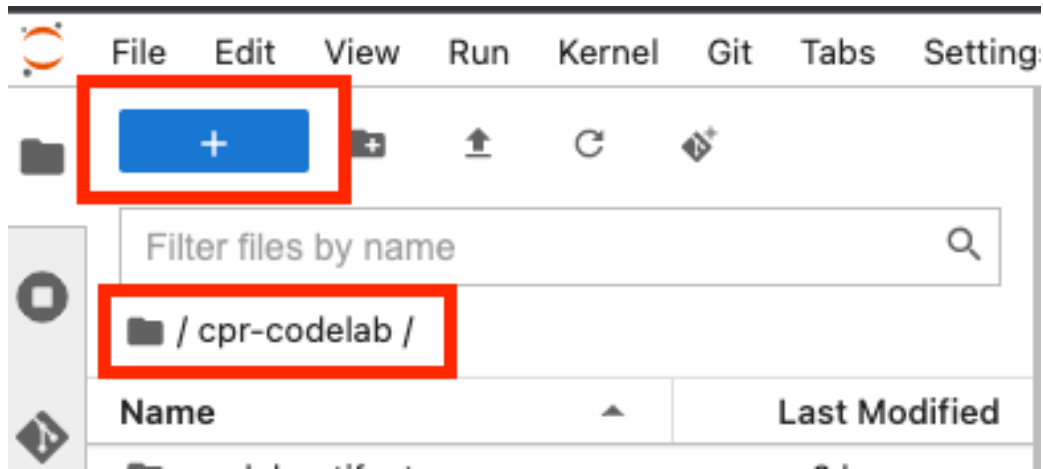
```
BUCKET="gs://${PROJECT_ID}-cpr-bucket"  
gsutil mb -l us-central1 $BUCKET
```

步驟 2：訓練模型

在終端機中建立名為 `cpr-codelab` 的新目錄，然後 `cd` 到該目錄。

```
mkdir cpr-codelab  
cd cpr-codelab
```

在檔案瀏覽器中，前往新的 `cpr-codelab` 目錄，然後使用啟動器建立名為 `task.ipynb` 的新 Python 3 筆記本。



您的 cpr-codelab 目錄現在應如下所示：

```
+ cpr-codelab/  
  + task.ipynb
```

在筆記本中貼上下列程式碼。

首先，請編寫 requirements.txt 檔案。

```
%%writefile requirements.txt  
fastapi  
uvicorn==0.17.6  
joblib~=1.1.1  
numpy>=1.17.3, <1.24.0  
scikit-learn~=1.0.0  
pandas  
google-cloud-storage>=2.2.1,<3.0.0dev  
google-cloud-aiplatform[prediction]>=1.18.2
```

您部署的模型預先安裝的依附元件，與筆記本環境不同。因此，您需要在 requirements.txt 中列出模型的所有依附元件，然後使用 pip 在筆記本中安裝完全相同的依附元件。稍後，您將在本機測試模型，然後部署至 Vertex AI，再次確認環境是否相符。

Pip 會在筆記本中安裝依附元件。

```
!pip install -U --user -r requirements.txt
```

請注意，pip 安裝完成後，您必須重新啟動核心。

接著，請建立目錄，用於儲存模型和前處理構件。

```
USER_SRC_DIR = "src_dir"  
!mkdir $USER_SRC_DIR  
!mkdir model_artifacts  
  
# copy the requirements to the source dir  
!cp requirements.txt $USER_SRC_DIR/requirements.txt
```

您的 cpr-codelab 目錄現在應如下所示：

```
+ cpr-codelab/  
  + model_artifacts/  
  + scr_dir/  
    + requirements.txt  
  + task.ipynb  
  + requirements.txt
```

目錄結構設定完成後，就可以開始訓練模型了！

首先，匯入程式庫。

```
import seaborn as sns  
import numpy as np  
import pandas as pd  
  
from sklearn import preprocessing  
from sklearn.ensemble import RandomForestRegressor  
from sklearn.pipeline import make_pipeline  
from sklearn.compose import make_column_transformer  
  
import joblib  
import logging  
  
# set logging to see the docker container logs  
logging.basicConfig(level=logging.INFO)
```

然後定義下列變數。請務必將 PROJECT_ID 替換為專案 ID，並將 BUCKET_NAME 替換為您在上一個步驟中建立的區域。

```
REGION = "us-central1"  
MODEL_ARTIFACT_DIR = "sklearn-model-artifacts"  
REPOSITORY = "diamonds"  
IMAGE = "sklearn-image"  
MODEL_DISPLAY_NAME = "diamonds-cpr"  
  
# Replace with your project  
PROJECT_ID = "{PROJECT_ID}"  
  
# Replace with your bucket  
BUCKET_NAME = "gs://{BUCKET_NAME}"
```

從 Seaborn 程式庫載入資料，然後建立兩個資料架構，一個包含特徵，另一個包含標籤。

```
data = sns.load_dataset('diamonds', cache=True, data_home=None)

label = 'price'

y_train = data['price']
x_train = data.drop(columns=['price'])
```

我們來看看訓練資料。您會看到每一列都代表一個鑽石。

```
x_train.head()
```

以及對應的價格標籤。

```
y_train.head()
```

現在，請定義 `sklearn` [資料欄轉換](#)，對類別特徵進行 one-hot 編碼，並縮放數值特徵

```
column_transform = make_column_transformer(
    (preprocessing.OneHotEncoder(sparse=False), [1,2,3]),
    (preprocessing.StandardScaler(), [0,4,5,6,7,8]))
```

定義隨機樹系模型

```
regr = RandomForestRegressor(max_depth=10, random_state=0)
```

接著，建立 `sklearn` [管道](#)。也就是說，傳送至這個管道的資料會先經過編碼/縮放，再傳送至模型。

```
my_pipeline = make_pipeline(column_transform, regr)
```

使用訓練資料擬合管道

```
my_pipeline.fit(x_train, y_train)
```

請試用模型，確保模型運作正常。對模型呼叫預測方法，並傳入測試樣本。

```
my_pipeline.predict([[0.23, 'Ideal', 'E', 'SI2', 61.5, 55.0, 3.95, 3.98, 2.43]])
```

現在我們可以將管道儲存至 `model_artifacts` 目錄，並複製到 Cloud Storage bucket

```
joblib.dump(my_pipeline, 'model_artifacts/model.joblib')

!gsutil cp model_artifacts/model.joblib {BUCKET_NAME}/{MODEL_ARTIFACT_DIR}/
```

步驟 3：儲存前處理構件

接下來，您將建立前處理構件。模型伺服器啟動時，這個構件會載入自訂容器。前處理構件幾乎可以是任何形式 (例如 Pickle 檔案)，但在此情況下，您會將字典寫入 JSON 檔案。

```
clarity_dict={"Flawless": "FL",
              "Internally Flawless": "IF",
              "Very Very Slightly Included": "VVS1",
              "Very Slightly Included": "VS2",
              "Slightly Included": "S12",
              "Included": "I3"}
```

訓練資料中的淨度特徵一律採用縮寫形式 (例如「FL」而非「無瑕」)。在放送時，我們希望檢查這項功能的資料是否也經過縮寫。這是因為模型知道如何對「FL」進行單熱編碼，但不知道如何對「Flawless」進行單熱編碼。您稍後會編寫這項自訂前處理邏輯。但目前只要將這個對照表儲存至 JSON 檔案，然後寫入 Cloud Storage bucket 即可。

```
import json
with open("model_artifacts/preprocessor.json", "w") as f:
    json.dump(clarity_dict, f)

!gsutil cp model_artifacts/preprocessor.json {BUCKET_NAME}/{MODEL_ARTIFACT_DIR}/
```

您本機的 cpr-codelab 目錄現在應如下所示：

```
+ cpr-codelab/
  + model_artifacts/
    + model.joblib
    + preprocessor.json
  + scr_dir/
    + requirements.txt
  + task.ipynb
  + requirements.txt
```

8. 使用 CPR 模型伺服器建構自訂服務容器

模型訓練完成並儲存預先處理構件後，現在可以建構自訂服務容器。通常建立供應容器時，需要撰寫模型伺服器程式碼。不過，使用自訂預測處理常式時，Vertex AI Predictions 會為您產生模型伺服器，並建構自訂容器映像檔。

自訂放送容器包含下列 3 段程式碼：

1. 模型伺服器 (SDK 會自動產生這個伺服器，並儲存在 `scr_dir/` 中)

- 代管模型的 HTTP 伺服器
- 負責設定路徑/連接埠等。

2. 要求處理常式

- 負責處理要求的網路伺服器相關作業，例如將要求主體反序列化、將回應序列化、設定回應標頭等。
- 在本例中，您將使用 SDK 中提供的預設處理常式 `google.cloud.aiplatform.prediction.handler.PredictionHandler`。

3. 預測器

- 負責處理預測要求的機器學習邏輯。

您可以根據用途需求自訂這些元件。在本範例中，您只會實作預測器。

預測者負責處理預測要求的機器學習邏輯，例如自訂預先處理和後續處理。如要編寫自訂預測邏輯，請將 [Vertex AI Predictor](#) 介面子類別化。

這個版本的自訂預測常式隨附可重複使用的 XGBoost 和 Sklearn 預測器，但如果您需要使用其他框架，可以透過子類別化基礎預測器來建立自己的預測器。

請參閱下方的 [Sklearn 預測器](#) 範例。如要建構這個自訂模型伺服器，您只需要編寫這些程式碼。

```
class SklearnPredictor(Predictor):
    """Default Predictor implementation for Sklearn models."""

    def __init__(self):
        return

    def load(self, artifacts_uri: str):
        prediction_utils.download_model_artifacts(artifacts_uri)
        self._model = joblib.load("model.joblib")

    def preprocess(self, prediction_input: dict) -> np.ndarray:
        instances = prediction_input["instances"]
        return np.asarray(instances)

    def predict(self, instances: np.ndarray) -> np.ndarray:
        return self._model.predict(instances)

    def postprocess(self, prediction_results: np.ndarray) -> dict:
        return {"predictions": prediction_results.tolist()}
```

在筆記本中貼上下列程式碼，將 `SklearnPredictor` 子類別化，並寫入 `src_dir/` 中的 Python 檔案。請注意，在這個範例中，我們只會自訂載入、預先處理和後續處理方法，不會自訂預測方法。

```
%%writefile $USER_SRC_DIR/predictor.py

import joblib
import numpy as np
import json

from google.cloud import storage
from google.cloud.aiplatform.prediction.sklearn.predictor import SklearnPredictor

class CprPredictor(SklearnPredictor):

    def __init__(self):
        return

    def load(self, artifacts_uri: str) -> None:
        """Loads the sklearn pipeline and preprocessing artifact."""

        super().load(artifacts_uri)

        # open preprocessing artifact
        with open("preprocessor.json", "rb") as f:
            self._preprocessor = json.load(f)

    def preprocess(self, prediction_input: np.ndarray) -> np.ndarray:
        """Performs preprocessing by checking if clarity feature is in abbreviated form."""

        inputs = super().preprocess(prediction_input)

        for sample in inputs:
            if sample[3] not in self._preprocessor.values():
                sample[3] = self._preprocessor[sample[3]]
        return inputs

    def postprocess(self, prediction_results: np.ndarray) -> dict:
        """Performs postprocessing by rounding predictions and converting to str."""

        return {"predictions": [f"${value}" for value in np.round(prediction_results)]}
```

接著我們將深入探討這些方法。

- 載入方法會載入前處理構件，在本例中，前處理構件是將鑽石淨度值對應至縮寫的字典。
- 前處理方法會使用該構件，確保在服務期間，清晰度功能會採用縮寫格式。如果不是，系統會將完整字串轉換為縮寫。
- 後續處理方法會傳回預測值，並以 \$ 符號表示，且會將值四捨五入。

接著，請使用 Vertex AI Python SDK 建構圖片。使用自訂預測處理常式時，系統會為您產生 Dockerfile 並建構映像檔。

```

from google.cloud import aiplatform

aiplatform.init(project=PROJECT_ID, location=REGION)

import os

from google.cloud.aiplatform.prediction import LocalModel

from src_dir.predictor import CprPredictor # Should be path of variable $USER_SRC_DIR

local_model = LocalModel.build_cpr_model(
    USER_SRC_DIR,
    f"{REGION}-docker.pkg.dev/{PROJECT_ID}/{REPOSITORY}/{IMAGE}",
    predictor=CprPredictor,
    requirements_path=os.path.join(USER_SRC_DIR, "requirements.txt"),
)

```

編寫測試檔，其中包含兩個用於預測的樣本。其中一個執行個體有縮寫的清晰度名稱，但另一個需要先轉換。

```

import json

sample = {"instances": [
    [0.23, 'Ideal', 'E', 'VS2', 61.5, 55.0, 3.95, 3.98, 2.43],
    [0.29, 'Premium', 'J', 'Internally Flawless', 52.5, 49.0, 4.00, 2.13, 3.11]]}

with open('instances.json', 'w') as fp:
    json.dump(sample, fp)

```

部署本機模型，在本機測試容器。

```

with local_model.deploy_to_local_endpoint(
    artifact_uri = 'model_artifacts/', # local path to artifacts
) as local_endpoint:
    predict_response = local_endpoint.predict(
        request_file='instances.json',
        headers={"Content-Type": "application/json"},
    )

    health_check_response = local_endpoint.run_health_check()

```

您可以透過下列方式查看預測結果：

```

predict_response.content

```

步驟 9 · 約 10 分鐘

9. 將模型部署至 Vertex AI

您已在本機測試容器，現在要將映像檔推送至 Artifact Registry，並將模型上傳至 Vertex AI Model Registry。

首先，請設定 Docker 來存取 Artifact Registry。

```
!gcloud artifacts repositories create {REPOSITORY} --repository-format=docker \
--location=us-central1 --description="Docker repository"

!gcloud auth configure-docker {REGION}-docker.pkg.dev --quiet
```

然後推送映像檔。

```
local_model.push_image()
```

然後上傳模型。

```
model = aiplatform.Model.upload(local_model = local_model,
                                display_name=MODEL_DISPLAY_NAME,
                                artifact_uri=f"{BUCKET_NAME}/{MODEL_ARTIFACT_DIR}",)
```

模型上傳完畢後，您應該會在控制台中看到模型：

接著部署模型，即可用於線上預測。自訂預測處理常式也適用於批次預測，因此如果您的用途不需要線上預測，就不必部署模型。

然後推送映像檔。

```
endpoint = model.deploy(machine_type="n1-standard-2")
```

最後，取得預測結果，測試已部署的模型。

```
endpoint.predict(instances=[[0.23, 'Ideal', 'E', 'VS2', 61.5, 55.0, 3.95, 3.98, 2.43]])
```

🎉 恭喜！🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

- 使用自訂預測處理常式，撰寫自訂預先處理和後續處理邏輯

Cosmopup 認為程式碼研究室很棒！



後續步驟

延伸閱讀和影片

- [什麼是 Vertex AI ?](#)
- [開始使用 Vertex AI](#)
- [我適合使用 Vertex AI 的哪種 AI/機器學習解決方案？](#)
- [使用 Vertex AI 建構問答系統](#)

參考文件

- [Vertex AI 簡介](#)
-